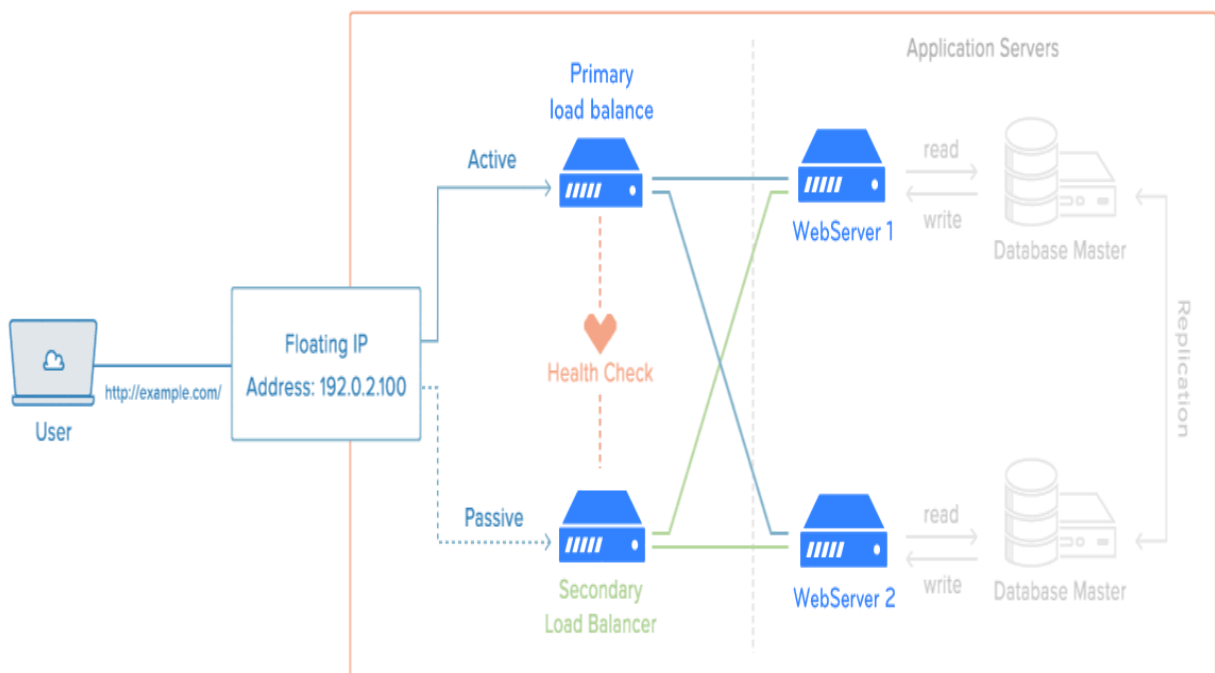


Load Balancing Using HAProxy Cluster (keepalived)

High availability allows an application to automatically restart or reroute work to another capable system in the event of a failure, there must be a component that can redirect the work and must be a mechanism to monitor for failure and transition the system if an interruption is detected.

The keepalived daemon can be used to monitor services or systems and to automatically failover to a standby if problems occur. We will configure a floating IP address that can be moved between two capable load balancers.

These will each be configured to split traffic between two backend web servers. If the primary load balancer goes down, the floating IP will be moved to the second load balancer automatically, allowing service to resume.



- 1 Active/Passive Cluster is healthy
- 2 Primary node fails
- 3 Floating IP is assigned to Secondary node

Tasks:

1. Hardware Recommendations
2. Installing HAproxy load balancer
3. Installing Keepalived (virtual IP)
4. Configuring Keepalived (virtual IP)
5. Configuring HAproxy as a load balancer
6. Configuring HAproxy Monitoring
7. Managing the HAproxy server
8. Configure hosts file
9. Test HAProxy using a web browser
10. Testing HAproxy Monitoring

STEP 1 :

Hardware Recommendations :

The hardware requirements for HAProxy Enterprise depend on the workload it needs to manage:

- Only CPU and memory are taken into consideration.
- Disk size depends on your operating system and the volume of logs you want to keep.

Workload	Tasks	Sizing
Low-level	• TCP or HTTP traffic • Up to 1000 conn/s • Very low SSL traffic or gzip compression	This type of workload can be achieved either by a Virtual Machine or a bare metal server. You need at least: • 1 CPU core • 1 G of RAM
Mid-level	• TCP or HTTP traffic (including HTTP manipulation) • Up to 4000 conn/s • Low SSL traffic or gzip compression	This type of workload can be achieved either by a Virtual Machine or a bare metal server. You need at least: • 2 CPU cores • 1 G of RAM
High-level	• TCP or HTTP traffic (including HTTP manipulation) • Up to 20000 conn/s • 10% of traffic ciphered (SSL) or compressed	This type of workload can be achieved by a bare metal server only. You need at least: • 2 CPU cores, as fast as possible • 4G of RAM • powerful network card

STEP 2 :

Installing HAproxy load balancer :

we will be installing the HAProxy on :

HAProxy-master: (x.x.21.108)

HAProxy-backup: (x.x.18.89)

To do so, update apt using the following command in Terminal :

Then update packages using the below command:

```
sudo apt-get update
```

```
sudo apt-get upgrade-
```

Now install HAproxy using the following command in Terminal :

```
sudo apt install haproxy
```

Once the installation of the HAproxy server is finished, you can confirm it using the below command in Terminal :

```
haproxy -v
```

It will show you the installed version of HAproxy on your system which verifies that the HAproxy has been successfully installed.

STEP 3 :

Installing Keepalived (Virtual IP)

we will be installing the Keepalived on

HAProxy-master: (x.x.21.108)

HAProxy-backup: (x.x.18.89)

To do so, update apt using the following command in Terminal :

Then update packages using the below command:

```
sudo apt-get update
```

```
sudo apt-get upgrade-
```

Now install keepalived using the following command in Terminal :

```
sudo apt install keepalived
```

Once the installation of the keepalived server is finished, you can confirm it using the below command in Terminal :

```
keepalived -v
```

It will show you the installed version of keepalived on your system which verifies that the keepalived has been successfully installed.

STEP 4 :

Configuring Keepalived (Virtual IP)

Node : HAProxy-master: (x.x.21.108)

Virtual IP : x.x.21.10

To allow HAProxy to bind to the shared IP address, we add the following line to /etc/sysctl.conf:

```
sudo nano /etc/sysctl.conf  
  
net.ipv4.ip_forward = 1  
net.ipv4.ip_nonlocal_bind = 1
```

: Check

```
sudo sysctl -p
```

At this stage, it is also worth making sure that HAProxy boots itself after rebooting :

```
sudo nano /etc/default/haproxy  
  
ENABLED=1
```

:Configure keepalived

configuration file : /etc/keepalived/keepalived.conf

```
sudo nano /etc/keepalived/keepalived.conf
```

```
global_defs {  
# Keepalived process identifier  
lvs_id haproxy_DH  
}  
# Script used to check if HAProxy is running  
vrrp_script check_haproxy {  
script "killall -0 haproxy"  
interval 2  
weight 2  
}  
# Virtual interface
```

```
# The priority specifies the order in which the assigned interface to take
over in a failover
vrrp_instance VI_01 {
state MASTER
interface eth0
virtual_router_id 51
priority 101
# The virtual ip address shared between the two loadbalancers
virtual_ipaddress {
172.17.21.10
}
track_script {
check_haproxy
}
}
```

sudo systemctl restart keepalived.service
sudo systemctl status keepalived.service

Check Virtual IP :
Ip addr sh eth0

Node : HAProxy-Backup: (x.x.18.89)

Virtual IP : x.x.21.10

To allow HAProxy to bind to the shared IP address, we add the following line to /etc/sysctl.conf:

```
sudo nano /etc/sysctl.conf

net.ipv4.ip_forward = 1
net.ipv4.ip_nonlocal_bind = 1
```

: Check

```
sysctl -p
```

At this stage, it is also worth making sure that HAProxy boots itself after rebooting :

```
sudo nano /etc/default/haproxy
```

```
ENABLED=1
```

:Configure keepalived

configuration file : /etc/keepalived/keepalived.conf

sudo nano /etc/keepalived/keepalived.conf

```
global_defs {
# Keepalived process identifier
lvs_id haproxy_DH
}
# Script used to check if HAProxy is running
vrrp_script check_haproxy {
script "killall -0 haproxy"
interval 2
weight 2
}
# Virtual interface
# The priority specifies the order in which the assigned interface to take
over in a failover
vrrp_instance VI_01 {
state BACKUP
interface eth0
virtual_router_id 51
priority 101
# The virtual ip address shared between the two loadbalancers
virtual_ipaddress {
172.17.21.10
}
track_script {
check_haproxy
}
}
```

sudo systemctl restart keepalived.service

sudo systemctl status keepalived.service

Check Virtual IP :

ip addr sh eth0

STEP 5 :

Configuring HAProxy as a load balancer :

Node : HAProxy-Master & Backup :

We will configure HAProxy as a load balancer. To do so, edit the `/etc/haproxy/haproxy.cfg` file :

```
sudo nano /etc/haproxy/haproxy.cfg
```

```
frontend http_front
```

```
    bind x.x.21.10:80
```

```
    stats uri /haproxy?stats
```

```
    default_backend x
```

```
backend xi
```

```
    balance roundrobin
```

```
    server worker1 x.x.56.3:32003 weight 1 check
```

```
    server worker2 x.x.56.4:32003 weight 1 check
```

STEP 4 :

Configuring HAproxy Monitoring :

With HAproxy monitoring, you can view a lot of information including server status, data transferred, uptime, session rate, etc. To configure HAproxy monitoring, append the following lines in the configuration file located at **/etc/haproxy/haproxy.cfg**:

```
sudo nano /etc/haproxy/haproxy.cfg
```

```
listen stats
bind x.x.21.10:8080
mode http
option forwardfor
option httpclose
stats enable
stats show-legends
stats refresh 5s
stats uri /stats
stats realm Haproxy\ Statistics
stats auth x:x      #Login User and Password for the monitoring
stats admin if TRUE
default_backend x
```

Once you are done with the configurations, save and close the **haproxy.cfg** file.

Now verify the configuration file using the below command in Terminal:

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

Now to apply the configurations, restart the HAproxy service:

```
sudo systemctl restart haproxy.service
```

STEP 5 :

Managing the HAproxy server: :

Here are some other commands for managing the HAproxy server:

In order to start the HAproxy server, the command would be:

```
sudo systemctl start haproxy.service
```

In order to stop the HAproxy server, the command would be:

```
sudo systemctl stop haproxy.service
```

In case you want to temporarily disable the HAproxy server, the command would be:

```
sudo systemctl disable haproxy.service
```

To re-enable the HAproxy server, the command would be :

```
sudo systemctl enable haproxy.service
```

STEP 6 :

Configure hosts file :

Edit the `/etc/hosts` file using the below command in Terminal:

```
sudo nano /etc/hosts
```

Add the following hostname entries for both Kubernetes workers along with its own hostname:

```
x.x.21.10 x
```

```
x.x.31.75 worker1
```

```
x.x.31.28 worker2
```

Now save and close the `/etc/hosts` file.

TEST Keepalived :

To test keepalived (VirtualIP) if we disable Haproxy service on any of the systems , VirtualIP should appear on the other system.

To check VirtualIP :

```
ip addr sh eth0
```

STEP 7 :

Test HAProxy using a web browser :

Now in your HAProxy server, open any web browser and type :

Add the following line in client host file :

```
172.17.21.10 x.local
```

```
http://x.local
```

STEP 8 :

Testing HAProxy Monitoring :

To access the HAProxy monitoring page, type `http://` followed by the IP address/hostname of HAProxy server and port 8080/stats:

```
http://x.x.21.10:8080/stats
```

username : x

password : x

This is the statistics report for our HAProxy server

Statistics Report for HAProxy

192.168.72.157:8080/stats

140%

HAProxy version 2.0.13-2ubuntu0.1, released 2020/09/08

Statistics Report for pid 5829

> General process information

pid = 5829 (process #1, nproc = 1, nbthread = 2)

uptime = 0d 0h00m11s

system limits: memmax = unlimited; ulimit-n = 1023

maxsock = 1023; maxconn = 490; maxpipes = 0

current conns = 1; current pipes = 0/0; conn rate = 1/sec; bit rate = 0.000 kbps

Running tasks: 1/18; idle = 100 %

active UP

active UP, going down

active DOWN, going up

active or backup DOWN

active or backup DOWN for maintenance (MAINT)

active or backup SOFT STOPPED for maintenance

backup UP

backup UP, going down

backup DOWN, going up

not checked

active or backup DOWN for maintenance (MAINT)

active or backup SOFT STOPPED for maintenance

Display option:

Scope :

Hide DOWN servers

Refresh now

CSV export

External resources:

Primary site

Updates (v2.0)

Online manual

Note: "NO LB"/"DRAIN" = UP with load-balancing disabled.

web-frontent

	Queue			Session rate			Sessions					Bytes		Denied		Errors		Warnings		Server										
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle	
Frontend	0	0	-	0	0	-	0	0	490	0			0	0	0	0	0	0	0	0	0	OPEN								

web-backend

	Queue			Session rate			Sessions					Bytes		Denied		Errors		Warnings		Server										
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle	
web-server1	0	0	-	0	1	0	1	-	1	1	4s	349	271	0	0	0	0	0	0	0	0	11s UP	L4OK in 0ms	1	Y	-	0	0	0s	-
web-server2	0	0	-	0	0	0	0	0	-	0	0	?	0	0	0	0	0	0	0	0	0	11s UP	L4OK in 1ms	1	Y	-	0	0	0s	-
Backend	0	0	0	1	0	1	98	1	1	4s	349	271	0	0	0	0	0	0	0	0	0	11s UP		2	2	0	0	0	0s	

stats

	Queue			Session rate			Sessions					Bytes		Denied		Errors		Warnings		Server										
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle	
Frontend	0	0	-	1	1	-	1	1	490	2			349	271	0	0	0	0	0	0	0	OPEN								
Backend	0	0	0	0	0	0	0	0	1	0	0s		0	0	0	0	0	0	0	0	0	11s UP		0	0	0		0		

Successful

END